RELATÓRIO TÉCNICO E DE DESIGN SYSTEM

Sistema Web de E-commerce Esportivo: SportZone

Integrantes: Yuri de Sousa Silva
Curso: Análise e Desenvolvimento de Sistemas – (PUC Goiás)

Introdução.

O SportZone é uma plataforma de e-commerce full-stack focada na venda de artigos esportivos. A aplicação foi projetada para simular um ambiente corporativo real, utilizando uma arquitetura baseada em Event-Driven Architecture para gerenciar o ciclo de vida dos pedidos. O sistema atende a diferentes perfis (Cliente, Administrador, Estoquista e Entregador) e foca em processamento assíncrono para garantir escalabilidade e resiliência.Problema que o Sistema Resolve

Em e-commerces tradicionais, o processamento síncrono de pedidos gera gargalos de performance e *timeouts*.

O SportZone resolve isso desacoplando: o cliente finaliza o pedido rapidamente (*Request/Response* imediato), e as validações pesadas (pagamento, baixa de estoque, roteamento de entrega) ocorrem em *background* via mensageria. O cliente é notificado do progresso em tempo real.

2. Design System (Guia de Estilos)

O SportZone adota um Design System proprietário com temática "Dark Mode", visando transmitir modernidade, energia e foco em performance, elementos essenciais no nicho esportivo.

2.1 Paleta de Cores

Nome	Hex	Uso
Dark Base	#0A0A0A	Fundo principal geral.
Dark Surface	#0F1115	Fundo de cards, painéis e modais.
Electric Orange	#F97316	Ação principal (CTA), *hover*, *badges* de destaque.
Orange Hover	#EA580C	Estado ativo/*hover* do CTA.

Text Primary	#FFFFFF	Títulos principais e textos de destaque.
Text Muted	#A1A1AA	Textos secundários e descrições.
Border / Line	#27272A	Divisores, bordas de *inputs* e cards.

2.2 Tipografia

- **Títulos e Destaques (Oswald / Inter - Black):** Utilizada em caixa alta (*uppercase*), com rastreamento reduzido (*tracking-tighter*) e leve inclinação para passar a ideia de esportivo.
- **Corpo e Leitura (Inter / Roboto - Regular/Bold):** Utilizada para descrições de produtos, preços e menus.

2.3 Componentes Principais

- **Botão Primário (CTA):** Fundo *Electric Orange*, texto em *Black*, característica do tema esportivo.
- **Cards de Produto:** Fundo *Dark Surface*, bordas visíveis que ganham destaque *Orange* no *hover*. Imagens dos produtos com efeito de zoom.
- **Badges de Status (Crucial para o Domínio):**
    - *Processando:* Cinza/Branco piscante (indicando aguardo na fila).
    - *Aprovado/Separando:* Laranja sólido.
    - *Enviado/Entregue:* Verde (#10B981).

3. Checklist de Boas Práticas UI/UX

Avaliação da interface com base nos critérios de usabilidade:

3.1 Hierarquia Visual e Layout

- **Hierarquia clara:** [ATENDE] O preço e o botão de "Comprar Agora" são os elementos de maior peso visual na tela de detalhes.
- **Espaçamento consistente:** [ATENDE] Uso de *paddings* e *margins* padronizados (escala de 4px/8px do Tailwind).
- **Densidade da interface:** [ATENDE] O *layout* escuro ajuda a focar nas imagens vibrantes dos produtos, sem poluição visual.

3.2 Feedback ao Usuário (Foco do Projeto)

- **Feedback Assíncrono (WebSockets):** [ATENDE] O diferencial do sistema. A tela de "Tracking" (*Dashboard* de Pedidos) atualiza o *status* visualmente (Processando -> Estoque -> Enviado) sem necessidade de *refresh*, refletindo o consumo das

mensagens no Kafka.

- **Mensagens de erro/sucesso:** [ATENDE] *Badges* e *toasts* claros para indisponibilidade de estoque ou sucesso na adição ao carrinho.
- **Estados vazios (*Empty States*):** [ATENDE] O Carrinho vazio exibe um ícone opaco com um CTA convidativo para voltar ao catálogo.

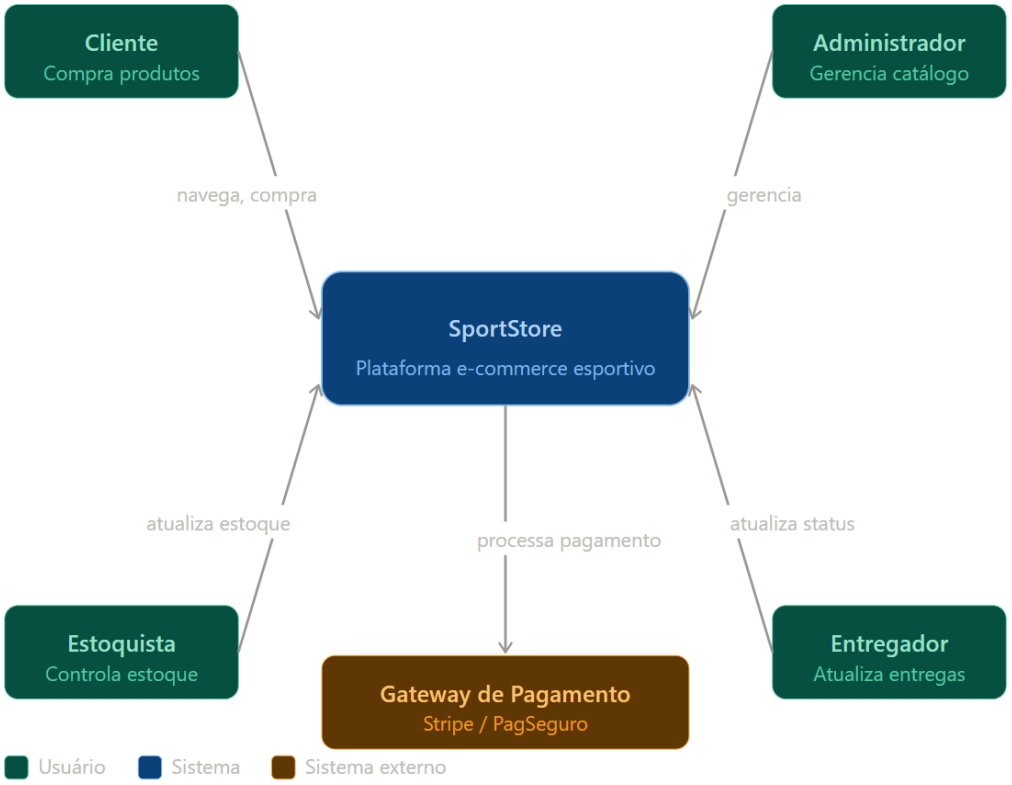
3.3 Acessibilidade e Responsividade

- **Funciona em mobile:** [ATENDE] O *layout* adota *Mobile First*, empilhando o carrinho e o formulário de *checkout* em telas menores.

4. Diagramas de Arquitetura (C4 Model)

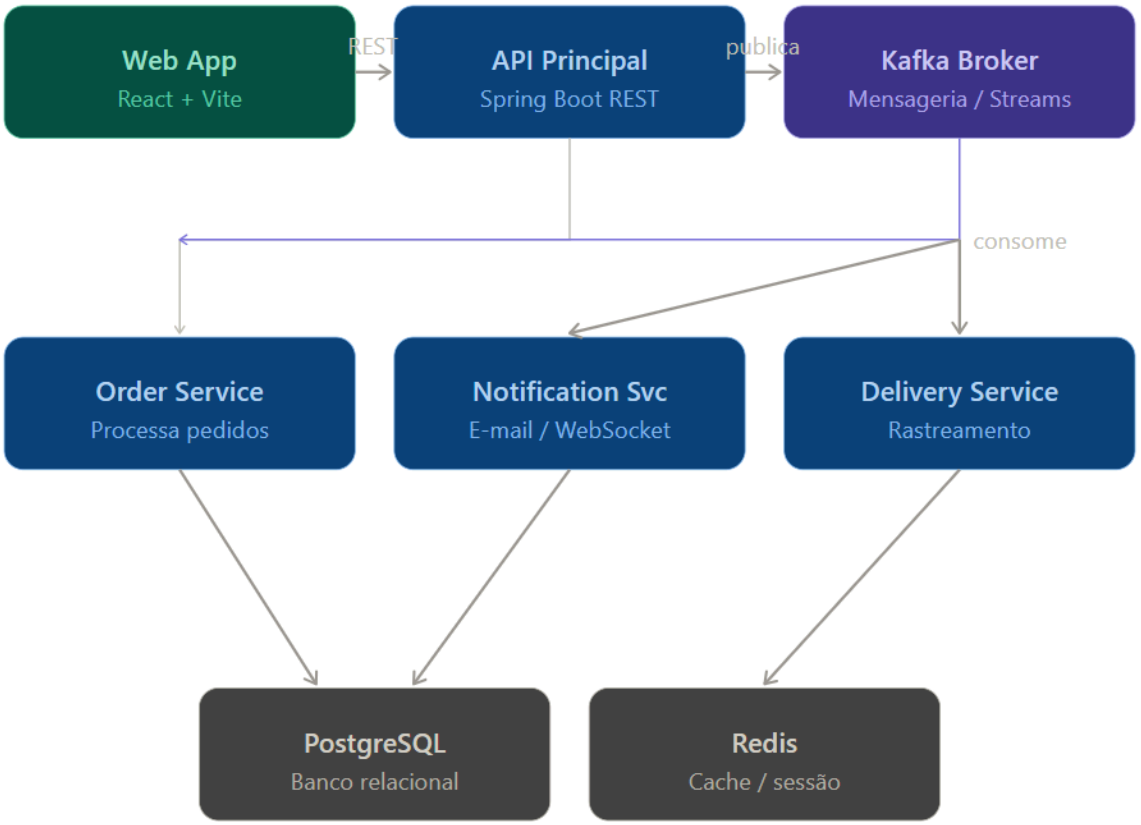
…

4.1 Diagrama de Contexto (Nível 1)



Apresenta o SportZone como o sistema central interagindo com Clientes (compradores), Administradores, Estoquistas e Entregadores, além da dependência externa do Gateway de Pagamento.

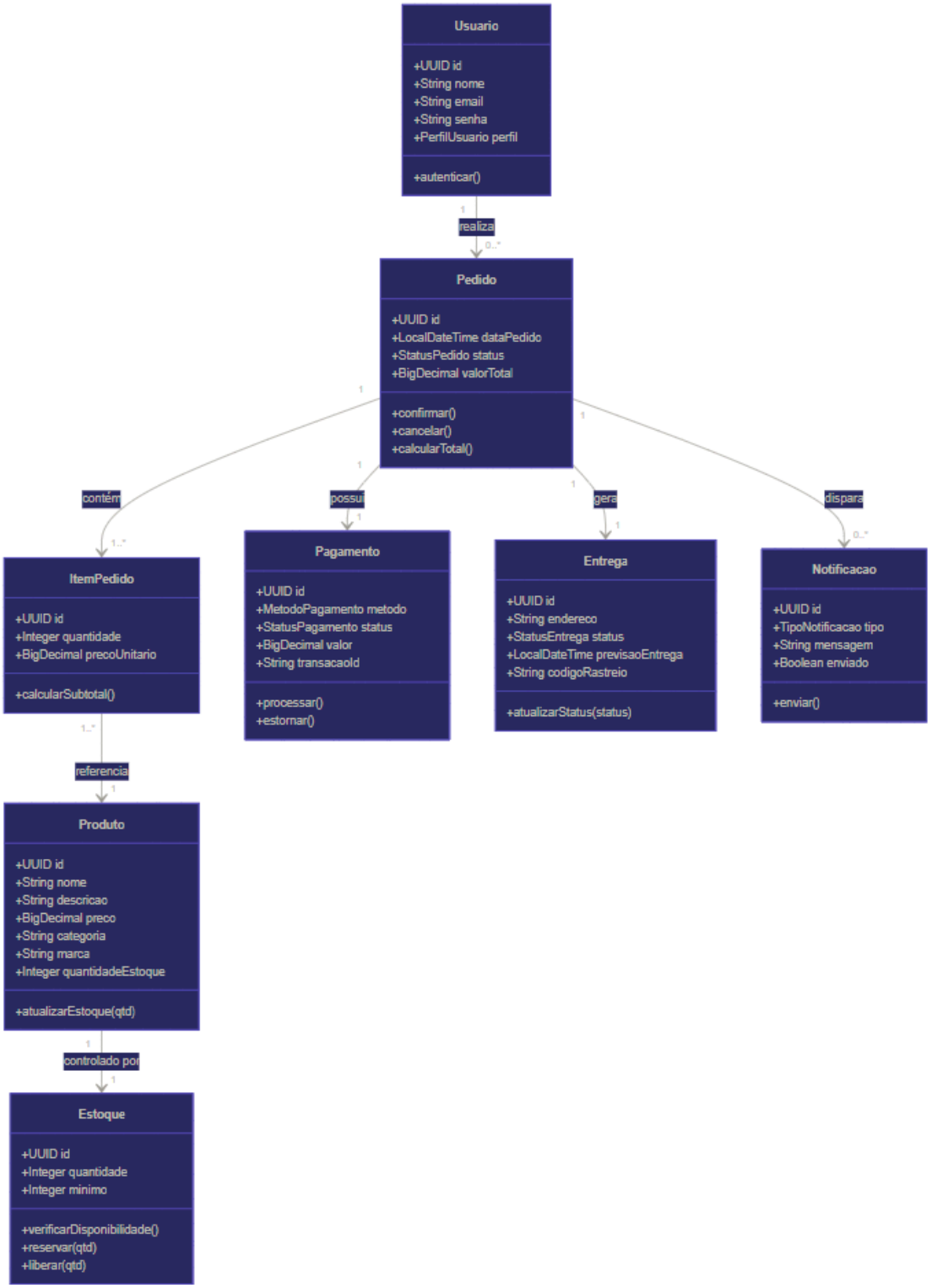
4.2 Diagrama de Container (Nível 2)



Detalha a divisão da aplicação Web (React) comunicando-se via REST com a API Principal (Spring Boot). Destaca o uso do **Kafka Broker** como canal de comunicação assíncrona, onde a API publica eventos (ex: *PedidoCriado*) e os módulos internos (Processamento, Notificação, Logística) os consomem e processam paralelamente.

5. Diagrama de Classes do Domínio

O domínio do SportZone foca na gestão de entidades vitais para um e-commerce.



- **Produto:** Contém informações base (nome, categoria) e delega o controle de quantidades para a entidade **Estoque**.

- **Pedido:** Entidade central. Composto por *ItemPedido* e fortemente ligado ao **Pagamento** e à **Entrega**. Possui uma relação de 1 para N com **Notificacao**, justificando o uso da mensageria para alertar o cliente a cada mudança de *StatusPedido*.

6. Padrões de Projeto Aplicados (Design Patterns)

Para garantir um código limpo (*Clean Code*) e fácil manutenção, utilizamos:

- **Strategy (Comportamental):**
  - *Problema:* O sistema aceita múltiplas formas de pagamento (Cartão, Pix, Boleto), cada uma com lógicas de processamento e taxas diferentes.
  - *Solução:* Uma interface `EstrategiaPagamento` com implementações específicas (`PagamentoPix`, `PagamentoCartao`). O serviço de pedidos apenas chama o método `processar()` sem precisar de dezenas de blocos *if/else*.
- **State (Comportamental):**
  - *Problema:* O ciclo de vida do pedido (Pendente -> Pago -> Separando -> Enviado) possui regras rígidas (um pedido enviado não pode voltar para pendente).
  - *Solução:* O padrão *State* controla as transições válidas de *status*, disparando os eventos para o Kafka a cada mudança de estado.
- **Observer (Arquitetural):**
  - *Implementação:* Padrão base da nossa mensageria. A API atua como

    Publisher (publica o evento no Kafka) e os demais serviços (Estoque, Notificação) atuam como *Subscribers* (Consumidores).

7. Levantamento de Requisitos

Para suportar a arquitetura distribuída e o processamento em background, o escopo do SportZone foi delimitado nos seguintes requisitos.

7.1 Requisitos Funcionais (RF)

Descrevem as ações que o sistema deve permitir que os usuários realizem, focando no fluxo de compra e logística.

ID	Funcionalidade	Descrição	Ator
RF-01	Catálogo e Busca	Visualizar, pesquisar e filtrar produtos por categorias esportivas.	Cliente
RF-02	Carrinho de Compras	Adicionar itens, alterar quantidades e calcular subtotal.	Cliente

RF-03	Checkout Assíncrono	Enviar os dados de compra e endereço. O sistema deve aceitar a requisição imediatamente e enviar para a fila de processamento.	Cliente
RF-04	Rastreio em Tempo Real	Visualizar a mudança de status do pedido (Processando, Separando, Enviado, Entregue) em um dashboard dinâmico.	Cliente
RF-05	Gestão de Estoque	Atualizar o inventário. Ao registrar a separação de um item, o sistema deve publicar o evento de atualização de status.	Estoquista / A…
RF-06	Logística	Registrar o envio e a entrega do pedido, notificando a plataforma.	Entregador

7.2 Requisitos Não Funcionais (RNF)

Descrevem os critérios de qualidade, restrições tecnológicas e de arquitetura exigidos para a operação do sistema.

ID	Categoria	Descrição Técnica
RNF-01	Arquitetura (Mensageria)	O sistema deve obrigatoriamente utilizar um serviço de mensageria (Apache Kafka) para desacoplar a recepção do pedido do seu processamento.

RNF-02	Comunicação (Streams)	A interface Web deve receber as atualizações de status do pedido de forma assíncrona (via WebSockets/SSE), sem necessidade de recarregamento manual da página (*polling* ou *refresh*).
RNF-03	Resiliência	O fluxo de navegação do usuário não deve ser bloqueado caso os serviços de retaguarda (logística/estoque) falhem; o pedido deve permanecer na fila até a recuperação do serviço.
RNF-04	Stack Tecnológica	O backend deve ser desenvolvido em Java com Spring Boot (API REST) e o frontend em React.
RNF-05	Banco de Dados	Os dados transacionais e relacionais devem ser persistidos no PostgreSQL, utilizando as práticas do Spring Data JPA.
RNF-06	Usabilidade (UX)	A interface deve ser responsiva (*Mobile-First*), com tempo de resposta inferior a 200ms para interações no frontend e feedback visual claro para ações em andamento.
RNF-07	Segurança	As rotas de API protegidas devem utilizar autenticação stateless via tokens JWT, e as senhas devem ser armazenadas com criptografia (ex: BCrypt).